第 4 讲：栈——保存尚未完成的工作

0. 栈解决什么问题

考虑括号字符串：

```
([{}])
```

读到最后一个 `)` 时，它应该与哪个左括号匹配？

答案不是最早出现的 `(` 以外的任意括号，而是**最近一个尚未匹配的左括号**。

这种“最近加入的未完成任务最先处理”的规则就是：

```
Last In, First Out
```

栈不仅用于括号。它常常保存：

- 尚未匹配的符号；
- 尚未计算的操作数；
- 尚未完成的函数调用；
- 等待未来元素解决的问题；
- 撤销操作需要的历史状态。

---

1. 栈是规则，不是一种固定语法

栈是一种抽象数据类型，它规定可以怎样操作数据：

- `push(x)`：把元素压入栈顶；
- `pop()`：删除并返回栈顶元素；
- `peek()`：查看栈顶但不删除；
- `is_empty()`：判断是否为空。

栈可以用动态数组实现，也可以用链表实现。

Python 通常直接使用 `list`：

```
[python]
stack = []

stack.append(10)  # push
stack.append(20)

top = stack[-1]   # peek    20
x = stack.pop()   # pop       20
```

`append()` 和末尾 `pop()` 都是摊还或通常 `O(1)`。

不要使用 `pop(0)` 模拟栈，因为删除开头需要移动后面的元素，是 `O(n)`。

---

2. 栈的状态图

执行：

```
[python]
stack = []
stack.append("A")
stack.append("B")
stack.append("C")
stack.pop()
```

状态变化：

```
[]
[A]
[A, B]
[A, B, C]
[A, B]       # C
```

只能直接操作顶部，并不意味着不能读取 Python 列表中的其他位置；而是算法主动遵守栈的访问规则。

---

3. 用类封装一个栈

Python 已经可以用 `list` 当栈，但手写一个小类有助于理解 `class` 和 `self`：

```
[python]
class ArrayStack:
    def __init__(self):
        self._items = []

    def push(self, value):
        self._items.append(value)

    def pop(self):
        if not self._items:
            raise IndexError("pop from empty stack")
        return self._items.pop()

    def peek(self):
        if not self._items:
            raise IndexError("peek from empty stack")
        return self._items[-1]

    def is_empty(self):
        return len(self._items) == 0

    def __len__(self):
        return len(self._items)
```

创建两个对象：

```
[python]
a = ArrayStack()
b = ArrayStack()

a.push(1)
b.push(9)
```

`a._items` 和 `b._items` 属于不同对象。调用：

```
[python]
a.push(1)
```

Python 自动把 `a` 传给 `self`。

`_items` 前的单下划线表示“这是内部实现细节”的约定，并不会真正禁止外部访问。

---

4. 模式一：匹配最近的未完成元素

4.1 有效括号

规则：

- 左括号出现时，还不能判断它与谁匹配，因此压栈；
- 右括号出现时，它必须匹配栈顶；
- 最后栈必须为空。

```
[python]
def is_valid_parentheses(text):
    pairs = {
        ")": "(",
        "]": "[",
        "}": "{",
    }
    stack = []

    for ch in text:
        if ch in "([{":
            stack.append(ch)
        else:
            if not stack:
                return False

            if stack.pop() != pairs[ch]:
                return False

    return not stack
```

循环不变量：

> 处理当前位置之前，`stack` 从底到顶保存了此前尚未匹配的左括号；栈顶是最先需要被未来右括号匹配的那个。

4.2 为什么要先判断空栈

输入：

```
")"
```

如果直接 `stack.pop()` 会产生 `IndexError`。这不是语法问题，而是没有先确认算法状态允许弹栈。

4.3 为什么结尾还要检查空栈

输入：

```
"((("
```

遍历过程中没有遇到错误右括号，但仍有左括号未匹配，所以必须：

因此函数最后应执行 `return not stack`。

---

5. 模式二：栈保存可合并的中间结果

删除相邻重复字符

输入：

```
"abbaca"
```

过程：

```
a      -> [a]
b      -> [a,b]
b      ->          -> [a]
a      ->          -> []
c      -> [c]
a      -> [c,a]
```

结果为 `"ca"`。

```
[python]
def remove_adjacent_duplicates(text):
    stack = []

    for ch in text:
        if stack and stack[-1] == ch:
            stack.pop()
        else:
            stack.append(ch)

    return "".join(stack)
```

不变量：

> 处理完当前前缀后，`stack` 是这个前缀完全消除相邻重复字符后的结果。

注意条件顺序：

条件应写成 `if stack and stack[-1] == ch:`。

Python 的 `and` 会短路。只有 `stack` 非空时才计算 `stack[-1]`。

---

6. 模式三：表达式求值

逆波兰表达式把运算符放在操作数之后：

```
["2", "1", "+", "3", "*"]
```

对应：

\[(2+1)\times3=9 \]

读到数字时压栈；读到运算符时弹出两个操作数，计算后把结果压回。

```
[python]
def evaluate_rpn(tokens):
    stack = []

    for token in tokens:
        if token not in {"+", "-", "*", "/"}:
            stack.append(int(token))
            continue

        right = stack.pop()
        left = stack.pop()

        if token == "+":
            value = left + right
        elif token == "-":
            value = left - right
        elif token == "*":
            value = left * right
```

```
            else:
                value = abs(left) // abs(right)
                if (left < 0) != (right < 0):
                    value = -value

            stack.append(value)

        return stack[-1]
```

减法和除法的顺序非常重要。后弹出的才是左操作数：

```
left operator right
```

LeetCode 的该题要求除法向零截断。先对绝对值整除，再根据符号决定正负，可以避免浮点数参与。Python 直接对负数使用 `//` 是向下取整，语义不同：

```
[python]
-3 // 2 == -2
int(-3 / 2) == -1
```

---

7. 设计题：支持常量时间最小值的栈

普通栈可以 `O(1)` 压入和弹出，但如果每次用 `min(stack)` 查询最小值，需要 `O(n)`。

解决方法：为每个入栈元素同时保存“到这里为止的最小值”。

```
  5   2   4   1
5   2   2   1
```

可以用两个栈：

```
[python]
class MinStack:
    def __init__(self):
        self.values = []
        self.minimums = []

    def push(self, value):
        self.values.append(value)

        if not self.minimums:
            self.minimums.append(value)
        else:
            self.minimums.append(min(value, self.minimums[-1]))

    def pop(self):
        self.minimums.pop()
        return self.values.pop()

    def top(self):
        return self.values[-1]

    def get_min(self):
        return self.minimums[-1]
```

两个栈长度始终相同。`minimums[i]` 表示 `values[:i+1]` 的最小值。

所有操作都是 `O(1)`，但使用了 `O(n)` 额外空间。

也可以在一个栈中保存 `(value, current_min)`，原理相同。

---

8. 单调栈：保存还没找到答案的元素

问题：对每一天的温度，求还要等几天才会出现更高温度。

```
temperatures = [73, 74, 75, 71, 69, 72, 76, 73]
answer       = [ 1,  1,  4,  2,  1,  1,  0,  0]
```

当读到温度 `76` 时，它会同时解决此前多个尚未找到更高温度的日子。

栈中保存下标，而不是温度值，因为答案需要计算距离：

```
[python]
def daily_temperatures(temperatures):
    answer = [0] * len(temperatures)
    stack = []

    for today, temperature in enumerate(temperatures):
        while stack and temperatures[stack[-1]] < temperature:
            previous_day = stack.pop()
            answer[previous_day] = today - previous_day

        stack.append(today)

    return answer
```

循环结束时，栈中剩余下标右侧都没有更高温度，答案保持 `0`。

8.1 为什么是单调栈

从栈底到栈顶，对应温度保持非递增。任何比当前温度低的栈顶都会被弹出，直到重新满足这个性质。

8.2 为什么不是 $O(n^2)$

虽然 `for` 内部有 `while`，但每个下标：

- 只入栈一次；
- 最多出栈一次。

所以所有 `while` 执行次数加起来最多是 `n`，总时间为 `O(n)`。

这与最长连续序列一样：不能只看循环嵌套层数，要计算元素在整个算法中被处理的总次数。

---

9. 调用栈与递归

函数调用本身也使用栈。调用新函数时，系统保存：

- 返回到哪里；
- 局部变量；
- 当前执行状态。

递归不断压入新的调用帧，返回时再逐层弹出。递归过深会触发 Python 的递归深度限制。

以后学习树的 DFS 时，会看到：

- 递归版本使用系统调用栈；
- 迭代版本显式使用 Python 列表作为栈。

它们在结构上是相似的。

---

10. 什么时候想到栈

题目出现以下信号时，可以考虑栈：

但不要因为题目中有“顺序”就使用栈。核心是是否符合后进先出。

---

11. 常见错误

11.1 对空栈执行 `pop` 或读取 `[-1]`

```
[python]
if stack and stack[-1] == value:
    ...
```

11.2 弹出顺序错误

对于减法、除法等不满足交换律的运算，先弹出的是右操作数。

11.3 栈中保存了错误的信息

每日温度需要距离，所以保存下标；只保存温度无法直接计算相隔天数。

11.4 只在局部维护性质，没有检查最终状态

括号题遍历完还要确认栈为空。

11.5 把栈当队列

- 栈：末尾加入、末尾取出；
- 队列：一端加入、另一端取出。

队列将在下一讲使用 `collections.deque`。

---

12. 本讲检查表

1.
2.
3.
4.
5.
6.
7.
8.
9.
10.

13. 本讲总结

1. 栈是后进先出的访问规则，可以用 Python `list` 实现。
2. 栈经常保存“最近的尚未完成工作”。
3. 括号匹配、相邻消除和表达式求值是基本模式。
4. MinStack 通过同步保存历史最小值，把查询最小值降到 `O(1)`。

5. 单调栈保存尚未找到答案的元素，并维持单调性质。

6. 判断复杂度时，应计算每个元素总共入栈、出栈多少次。